

SMART LENDER

Project Description:

Smart Lender is a machine learning-powered web application designed to predict the creditworthiness of loan applicants, enabling banks and financial institutions to make faster, data-driven loan approval decisions. The platform leverages classification algorithms Decision Tree, Random Forest, K-Nearest Neighbours (KNN), and XGBoost to evaluate applicant data and determine the likelihood of loan repayment or default.

The application processes structured applicant inputs such as gender, marital status, education, employment status, income, loan amount, loan term, credit history, and property area. After training and evaluating all four models, the best-performing model (XGBoost at 94.7% training accuracy and 81.1% testing accuracy) is saved and integrated into a Flask web application for real-time prediction.

Built with Python and Flask and designed for deployment on IBM Cloud, Smart Lender provides a seamless user interface where applicants can submit their details and instantly receive an approval prediction. The system empowers financial analysts, credit officers, and banking professionals to reduce non-performing assets and improve the overall credit evaluation process with confidence.

Scenarios:

Scenario 1: Fast-Track Approval for Low-Risk Applicants

A bank credit officer enters the details of a salaried applicant with a good credit history and stable income. The model predicts loan approval with high confidence, allowing the officer to fast-track the application without manual review.

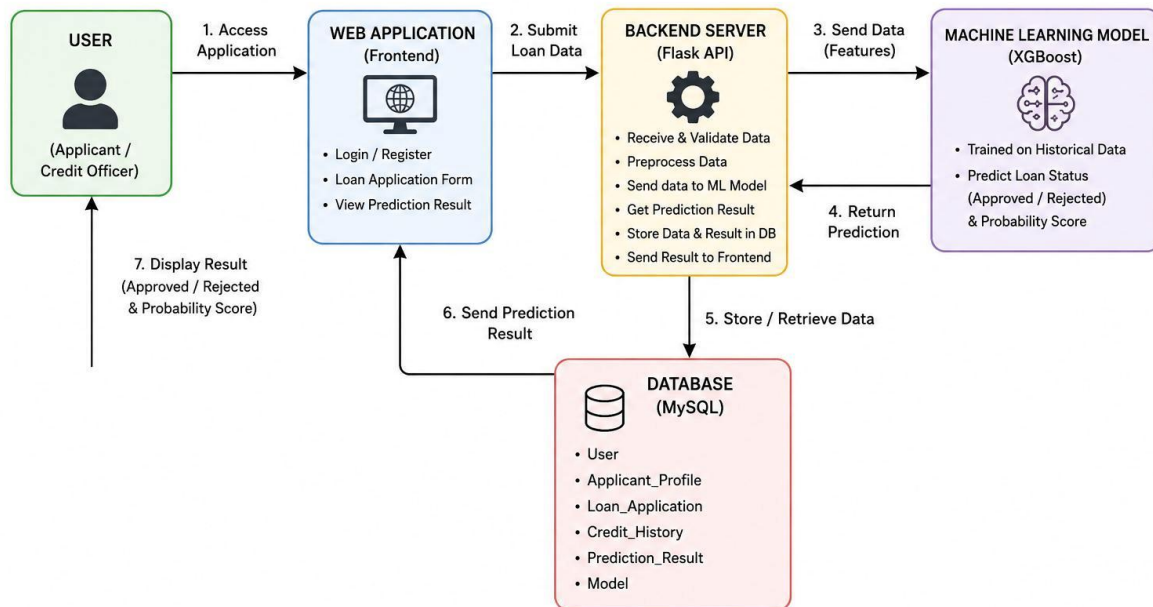
Scenario 2: High-Risk Applicant Detection

An applicant with irregular self-employment income and no credit history submits a loan request. The system flags the application as high-risk based on the trained model, prompting further scrutiny or document verification before proceeding.

Scenario 3: High-Risk Applicant Detection

A financial analyst uses the platform to batch-evaluate multiple applicants during a high-volume period. By relying on the XGBoost model's predictions, the analyst significantly reduces evaluation time while maintaining accuracy and compliance.

Technical Architecture:



Description: The technical architecture represents the overall workflow of the Smart Lender system. The user accesses the web application and submits loan details. The backend server processes the request and sends the data to the XGBoost machine learning model for prediction. The prediction result is stored in the database and returned to the web application, where the loan approval status is displayed to the user. This architecture ensures smooth communication between the user, application, machine learning model, and database.

Pre-requisites:

- Python 3.x
- Visual Studio Code (VS Code)
- Google Colab
- Flask
- Pandas
- NumPy
- Scikit-learn
- Pickle
- Web Browser (Chrome/Edge)

Project Workflow:

Milestone 1: Data Collection and Model Selection

- **Activity 1.1:** Collected the loan prediction dataset and analyzed the dataset features required for loan approval prediction.
- **Activity 1.2:** Performed data preprocessing by handling missing values, encoding categorical variables, and splitting the dataset into training and testing sets.
- **Activity 1.3:** Researched and selected suitable machine learning algorithms such as Decision Tree, Random Forest, and XGBoost for loan approval prediction.
- **Activity 1.4:** Set up the development environment using Python, Google Colab, VS Code, and installed the required libraries (Pandas, NumPy, Scikit-learn, Flask, XGBoost).

Milestone 2: Model Development and Training

- **Activity 2.1:** Trained and evaluated Decision Tree, Random Forest, and XGBoost models using the processed dataset.
- **Activity 2.2:** Compared the performance of the models and selected the best-performing model for loan approval prediction.
- **Activity 2.3:** Saved the trained machine learning model for integration with the web application.

Milestone 3: Flask Application Development

- **Activity 3.1:** Developed the Flask backend (app.py) to load the trained model and process user input.
- **Activity 3.2:** Implemented prediction logic to classify loan applications as approved or rejected based on the input details.

Milestone 4: Frontend Development

- **Activity 4.1:** Designed the user interface using HTML and CSS to collect applicant information.
- **Activity 4.2:** Connected the frontend with the Flask backend and displayed loan prediction results dynamically.
- **Activity 4.3:** Tested the application with different sample inputs to verify accurate predictions.

Milestone 5: Deployment and Testing

- **Activity 5.1:** Tested the complete Smart Lender application locally and verified all functionalities.
- **Activity 5.2:** Successfully deployed the application on Render and generated a public deployment URL.
- **Activity 5.3:** Validated the deployed application by testing multiple loan prediction scenarios and documented the results.

Milestone 6: Conclusion

Milestone 1: Data Collection and Model Selection

Activity 1.1: Collected the loan prediction dataset and analyzed the dataset features required for loan approval prediction.

AutoSave

loan_prediction - Saved to PC

Search

FileHomeInsertDrawPage LayoutFormulasDataReviewViewHelp

CutCopyFormat Painter

Clipboard

Aptos Narrow11A⁺

B

I

U

Text Color

Font Color

Font

Wrap Text

Merge & Center

Alignment

General

Number

Conditional Formatting

Format as Table

Cell Styles

Styles

Insert

Delete

Format

Cells

AutoSum

Fill

Clear

Editing

Comments

Share

Add-ins

Add-ins

POSSIBLE DATA LOSS

Some features might be lost if you save this workbook in the comma-delimited (.csv) format. To preserve these features, save it in an Excel file format.

Don't show again

Save As...

A1

fx

Loan_ID

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	Applicant_Income	Coapplicant_Income	Loan_Amount_Term	Loan_Amount_Term	Credit_History	Property_Area	Loan_Status			
2	LP001002	Male	No	0	Graduate	No	5849	0	360	1	Urban	Y				
3	LP001003	Male	Yes	1	Graduate	No	4583	1508	128	360	1	Rural	N			
4	LP001005	Male	Yes	0	Graduate	Yes	3000	0	66	360	1	Urban	Y			
5	LP001006	Male	Yes	0	Not Graduated	No	2583	2358	120	360	1	Urban	Y			
6	LP001008	Male	No	0	Graduate	No	6000	0	141	360	1	Urban	Y			
7	LP001011	Male	Yes	2	Graduate	Yes	5417	4196	267	360	1	Urban	Y			
8	LP001013	Male	Yes	0	Not Graduated	No	2333	1516	95	360	1	Urban	Y			
9	LP001014	Male	Yes	3+	Graduate	No	3036	2504	158	360	0	Semiurban	N			
10	LP001018	Male	Yes	2	Graduate	No	4006	1526	168	360	1	Urban	Y			
11	LP001020	Male	Yes	1	Graduate	No	12841	10968	349	360	1	Semiurban	N			
12	LP001024	Male	Yes	2	Graduate	No	3200	700	70	360	1	Urban	Y			
13	LP001027	Male	Yes	2	Graduate	No	2500	1840	109	360	1	Urban	Y			
14	LP001028	Male	Yes	2	Graduate	No	3073	8106	200	360	1	Urban	Y			
15	LP001029	Male	No	0	Graduate	No	1853	2840	114	360	1	Rural	N			
16	LP001030	Male	Yes	2	Graduate	No	1299	1086	17	120	1	Urban	Y			
17	LP001032	Male	No	0	Graduate	No	4950	0	125	360	1	Urban	Y			
18	LP001034	Male	No	1	Not Graduated	No	3596	0	100	240	Urban	Y				

loan_prediction

+

Ready

Accessibility: Unavailable

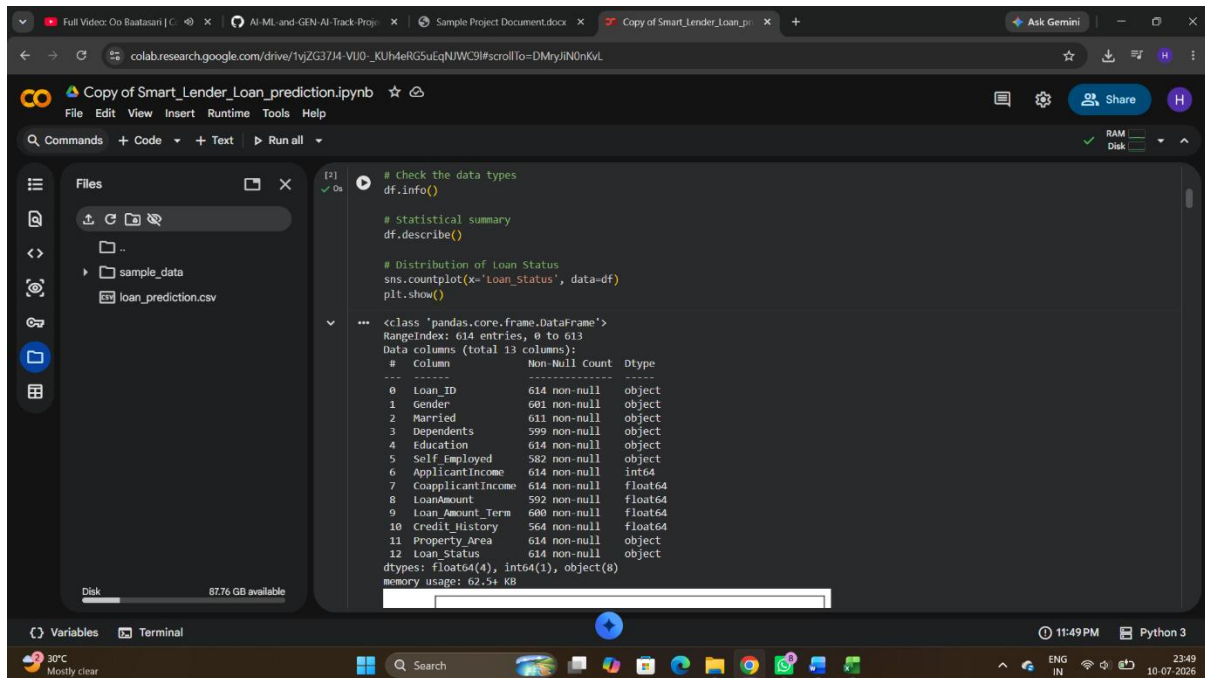
30°C Mostly clear

Search

ENG IN

10-07-2024

Activity 1.2: Performed data preprocessing by handling missing values, encoding categorical variables, and splitting the dataset into training and testing sets.



```

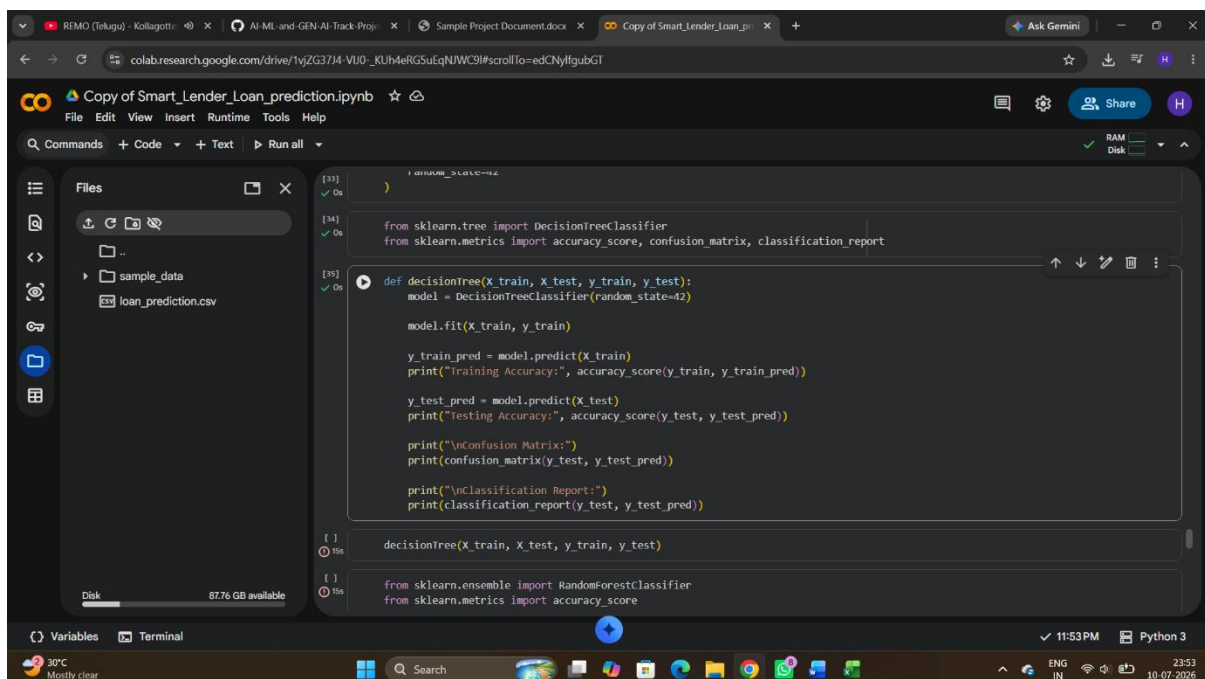
# Check the data types
df.info()

# Statistical summary
df.describe()

# Distribution of loan Status
sns.countplot(x='Loan_Status', data=df)
plt.show()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
 #   Column              Non-Null Count  Dtype  
---  --   ---
 0   Loan_ID              614 non-null    object  
 1   Gender                601 non-null    object  
 2   Married               611 non-null    object  
 3   Dependents            599 non-null    object  
 4   Education             614 non-null    object  
 5   Self_Employed         582 non-null    object  
 6   ApplicantIncome       614 non-null    int64   
 7   CoapplicantIncome     614 non-null    float64  
 8   LoanAmount            592 non-null    float64  
 9   Loan_Amount_Term      600 non-null    float64  
10   Credit_History        564 non-null    float64  
11   Property_Area         614 non-null    object  
12   Loan_Status          614 non-null    object  
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB
  
```

Activity 1.3: Researched and selected suitable machine learning algorithms such as Decision Tree, Random Forest, and XGBoost for loan approval prediction.



```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

def decisionTree(X_train, X_test, y_train, y_test):
    model = DecisionTreeClassifier(random_state=42)
    model.fit(X_train, y_train)

    y_train_pred = model.predict(X_train)
    print("Training Accuracy:", accuracy_score(y_train, y_train_pred))

    y_test_pred = model.predict(X_test)
    print("Testing Accuracy:", accuracy_score(y_test, y_test_pred))

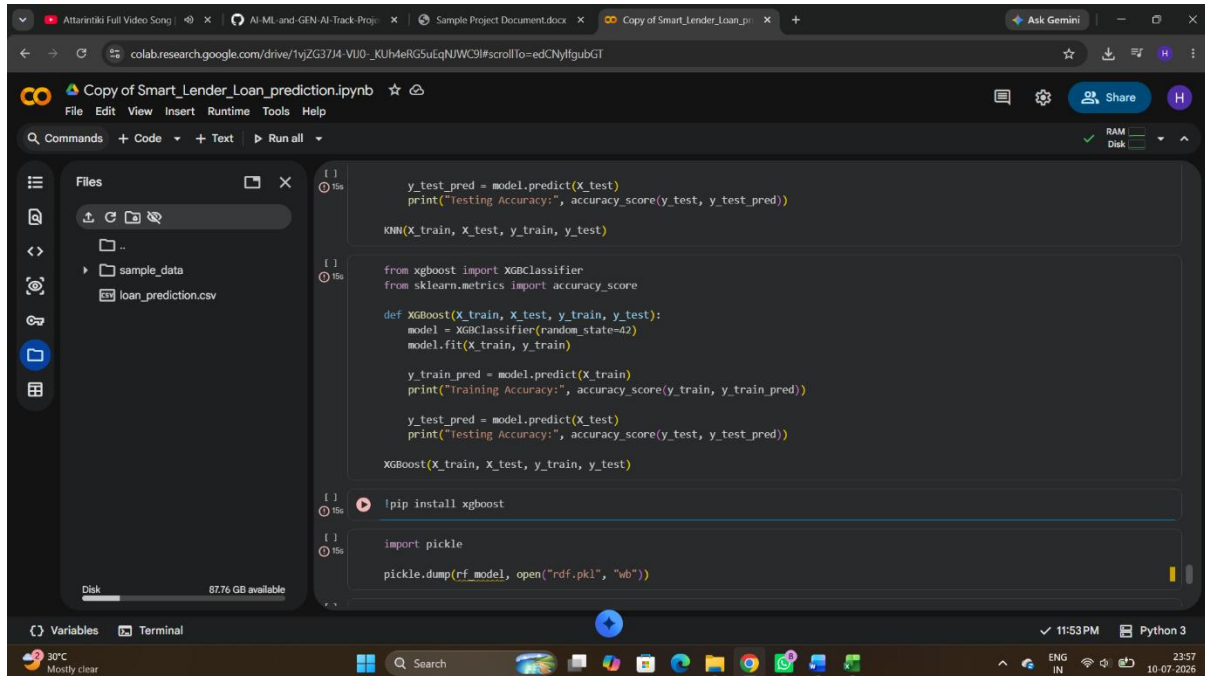
    print("\nConfusion Matrix:")
    print(confusion_matrix(y_test, y_test_pred))

    print("\nClassification Report:")
    print(classification_report(y_test, y_test_pred))

decisionTree(X_train, X_test, y_train, y_test)

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
  
```

Activity 1.4: Set up the development environment using Python, Google Colab, VS Code, and installed the required libraries (Pandas, NumPy, Scikit-learn, Flask, XGBoost).



The screenshot shows a Google Colab notebook titled "Copy of Smart_Lender_Loan_prediction.ipynb". The left sidebar shows a file explorer with "sample_data" and "loan_prediction.csv". The main code area contains the following Python code:

```

y_test_pred = model.predict(X_test)
print("Testing Accuracy:", accuracy_score(y_test, y_test_pred))

KNN(X_train, X_test, y_train, y_test)

from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score

def XGBoost(X_train, X_test, y_train, y_test):
    model = XGBClassifier(random_state=42)
    model.fit(X_train, y_train)

    y_train_pred = model.predict(X_train)
    print("Training Accuracy:", accuracy_score(y_train, y_train_pred))

    y_test_pred = model.predict(X_test)
    print("Testing Accuracy:", accuracy_score(y_test, y_test_pred))

XGBoost(X_train, X_test, y_train, y_test)

!pip install xgboost

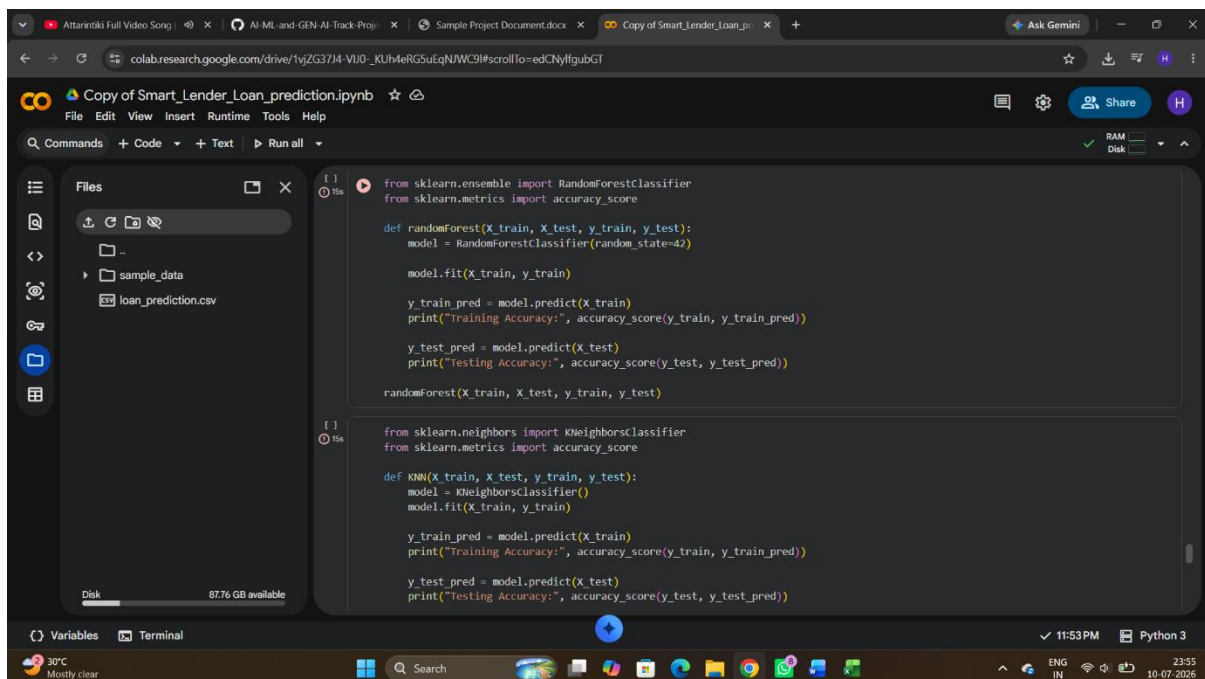
import pickle

pickle.dump(model, open("rf.pkl", "wb"))
  
```

The bottom status bar indicates "Python 3" and "11:53 PM".

Milestone 2: Model Development and Training

Activity 2.1: Trained and evaluated Decision Tree, Random Forest, and XGBoost models using the processed dataset.



The screenshot shows a Google Colab notebook titled "Copy of Smart_Lender_Loan_prediction.ipynb". The left sidebar shows a file explorer with "sample_data" and "loan_prediction.csv". The main code area contains the following Python code:

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

def randomForest(X_train, X_test, y_train, y_test):
    model = RandomForestClassifier(random_state=42)
    model.fit(X_train, y_train)

    y_train_pred = model.predict(X_train)
    print("Training Accuracy:", accuracy_score(y_train, y_train_pred))

    y_test_pred = model.predict(X_test)
    print("Testing Accuracy:", accuracy_score(y_test, y_test_pred))

randomForest(X_train, X_test, y_train, y_test)

from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

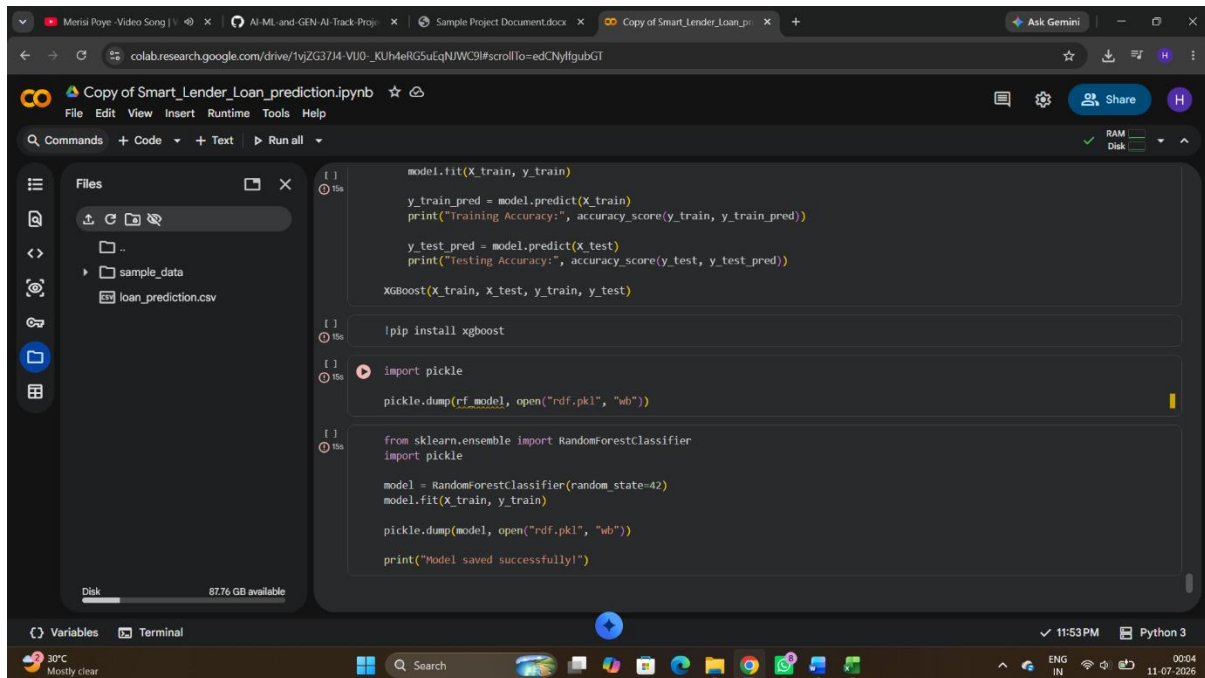
def KNN(X_train, X_test, y_train, y_test):
    model = KNeighborsClassifier()
    model.fit(X_train, y_train)

    y_train_pred = model.predict(X_train)
    print("Training Accuracy:", accuracy_score(y_train, y_train_pred))

    y_test_pred = model.predict(X_test)
    print("Testing Accuracy:", accuracy_score(y_test, y_test_pred))
  
```

The bottom status bar indicates "Python 3" and "11:53 PM".

Activity 2.2: Compared the performance of the models and selected the best-performing model for loan approval prediction.



The screenshot shows a Google Colab notebook titled "Copy of Smart_Lender_Loan_prediction.ipynb". The notebook is open to a cell containing the following Python code:

```

model.fit(X_train, y_train)

y_train_pred = model.predict(X_train)
print("Training Accuracy:", accuracy_score(y_train, y_train_pred))

y_test_pred = model.predict(X_test)
print("Testing Accuracy:", accuracy_score(y_test, y_test_pred))

XGBoost(X_train, X_test, y_train, y_test)

!pip install xgboost

import pickle
pickle.dump(rf_model, open("rdf.pkl", "wb"))

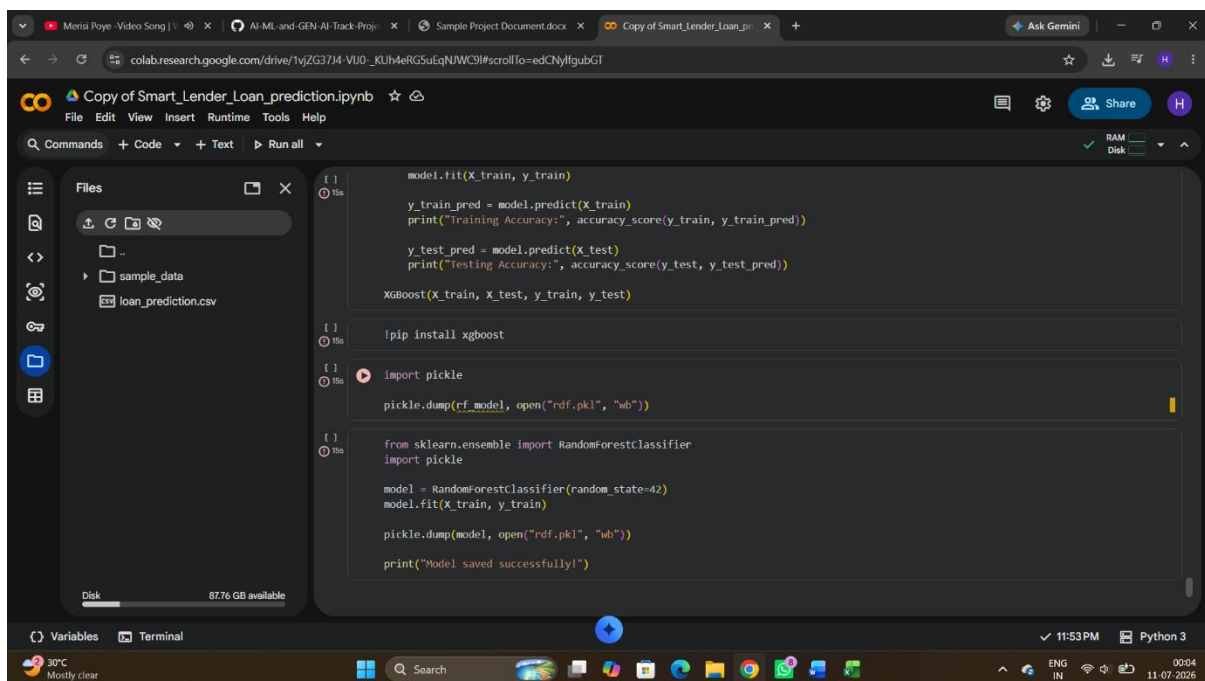
from sklearn.ensemble import RandomForestClassifier
import pickle

model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

pickle.dump(model, open("rdf.pkl", "wb"))
print("Model saved successfully!")
  
```

The notebook interface includes a file explorer on the left showing a folder named "sample_data" and a file named "loan_prediction.csv". The bottom status bar indicates "Python 3" and "11:53 PM".

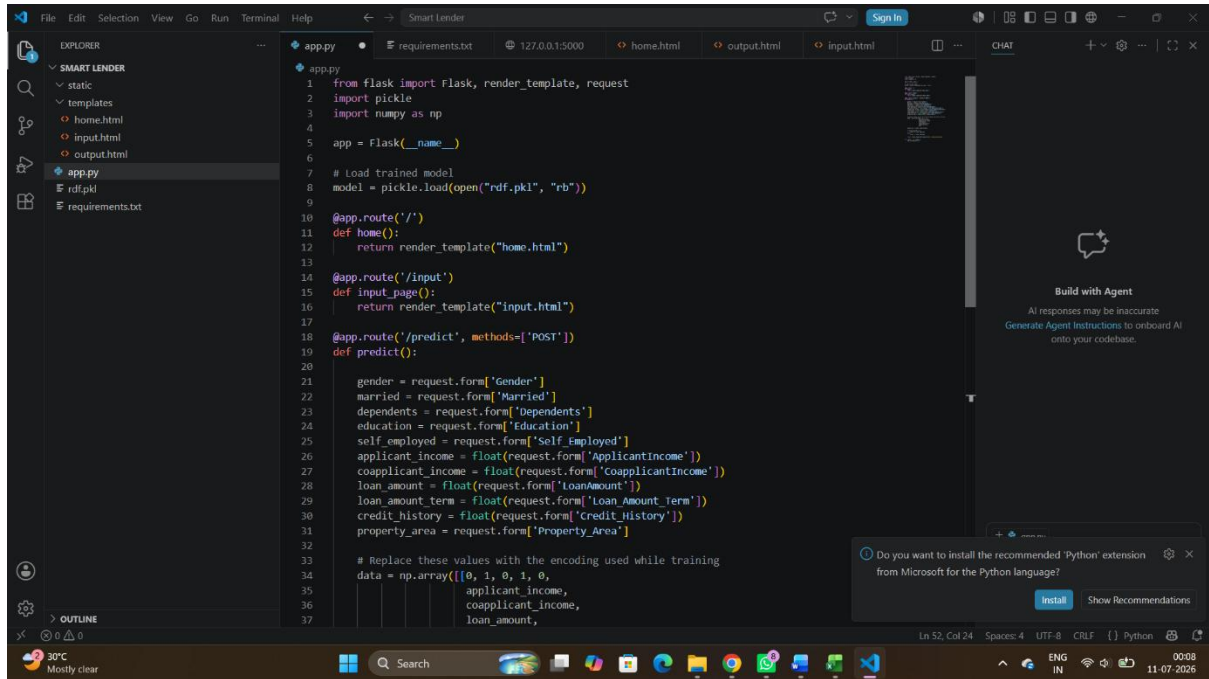
Activity 2.3: Saved the trained machine learning model for integration with the web application.



This screenshot is identical to the one above, showing the same Google Colab notebook with the same Python code for training and saving a machine learning model. The code includes training a model, calculating training and testing accuracy, and saving the model using pickle. The notebook interface and status bar are also identical.

Milestone 3: Flask Application Development

Activity 3.1: Developed the Flask backend (app.py) to load the trained model and process user input.

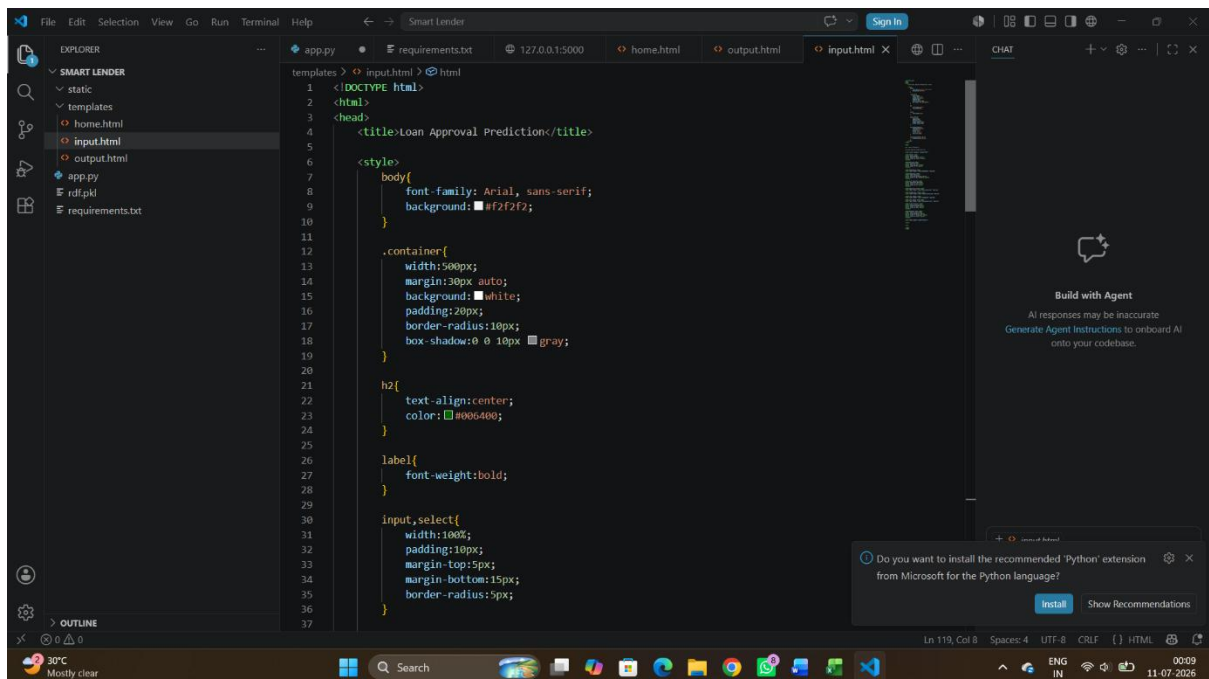


```

1  from flask import Flask, render_template, request
2  import pickle
3  import numpy as np
4
5  app = Flask(__name__)
6
7  # Load trained model
8  model = pickle.load(open("rdf.pkl", "rb"))
9
10 @app.route('/')
11 def home():
12     return render_template("home.html")
13
14 @app.route('/input')
15 def input_page():
16     return render_template("input.html")
17
18 @app.route('/predict', methods=['POST'])
19 def predict():
20
21     gender = request.form['Gender']
22     married = request.form['Married']
23     dependents = request.form['Dependents']
24     education = request.form['Education']
25     self_employed = request.form['Self_Employed']
26     applicant_income = float(request.form['ApplicantIncome'])
27     coapplicant_income = float(request.form['CoapplicantIncome'])
28     loan_amount = float(request.form['LoanAmount'])
29     loan_amount_term = float(request.form['Loan_Amount_Term'])
30     credit_history = float(request.form['Credit_History'])
31     property_area = request.form['Property_Area']
32
33     # Replace these values with the encoding used while training
34     data = np.array([[0, 1, 0, 1, 0,
35                     applicant_income,
36                     coapplicant_income,
37                     loan_amount,

```

Activity 3.2: Implemented prediction logic to classify loan applications as approved or rejected based on the input details.



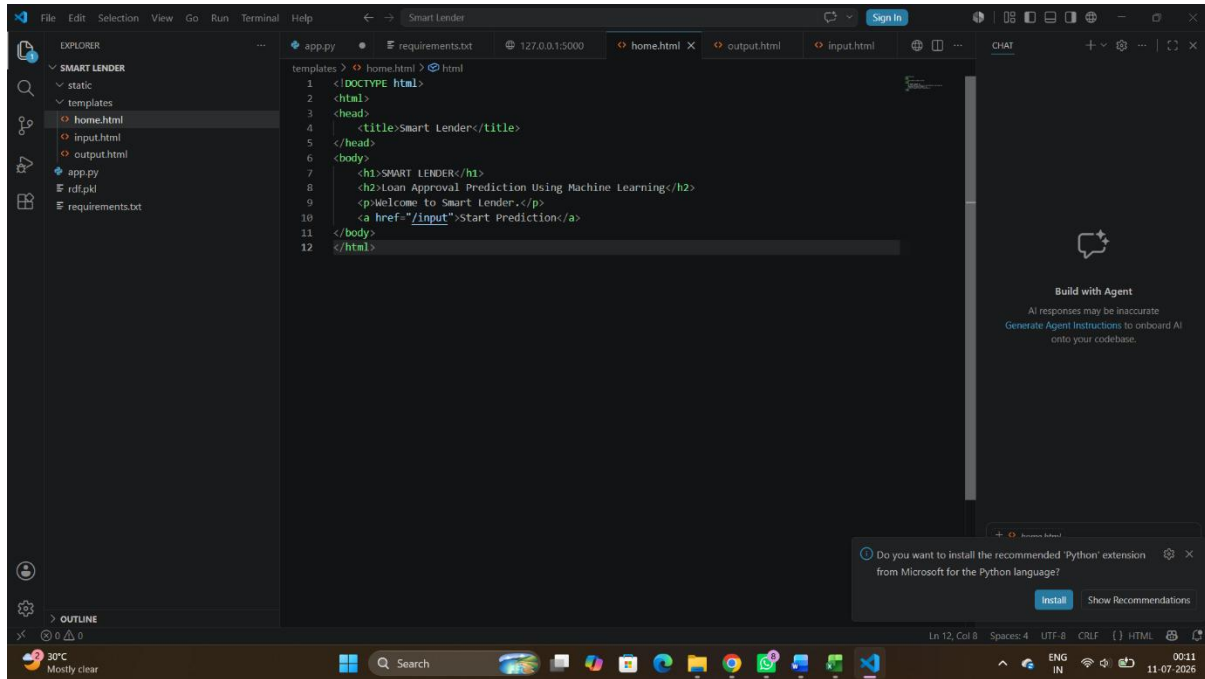
```

1  <!DOCTYPE html>
2  <html>
3  <head>
4  <title>Loan Approval Prediction</title>
5
6  <style>
7      body{
8          font-family: Arial, sans-serif;
9          background: #f2f2f2;
10     }
11
12     .container{
13         width:500px;
14         margin:30px auto;
15         background: #white;
16         padding:20px;
17         border-radius:10px;
18         box-shadow:0 0 10px #gray;
19     }
20
21     h2{
22         text-align:center;
23         color:#006400;
24     }
25
26     label{
27         font-weight:bold;
28     }
29
30     input,select{
31         width:100%;
32         padding:10px;
33         margin-top:5px;
34         margin-bottom:15px;
35         border-radius:5px;
36     }
37

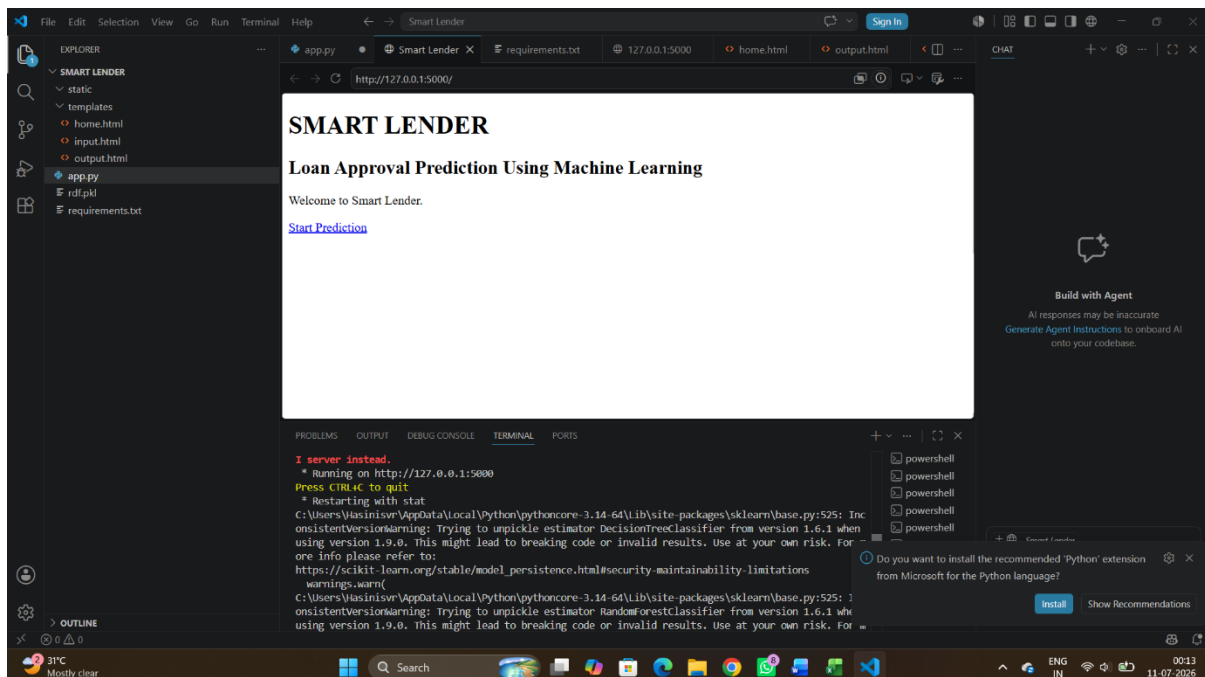
```


Milestone 4: Frontend Development

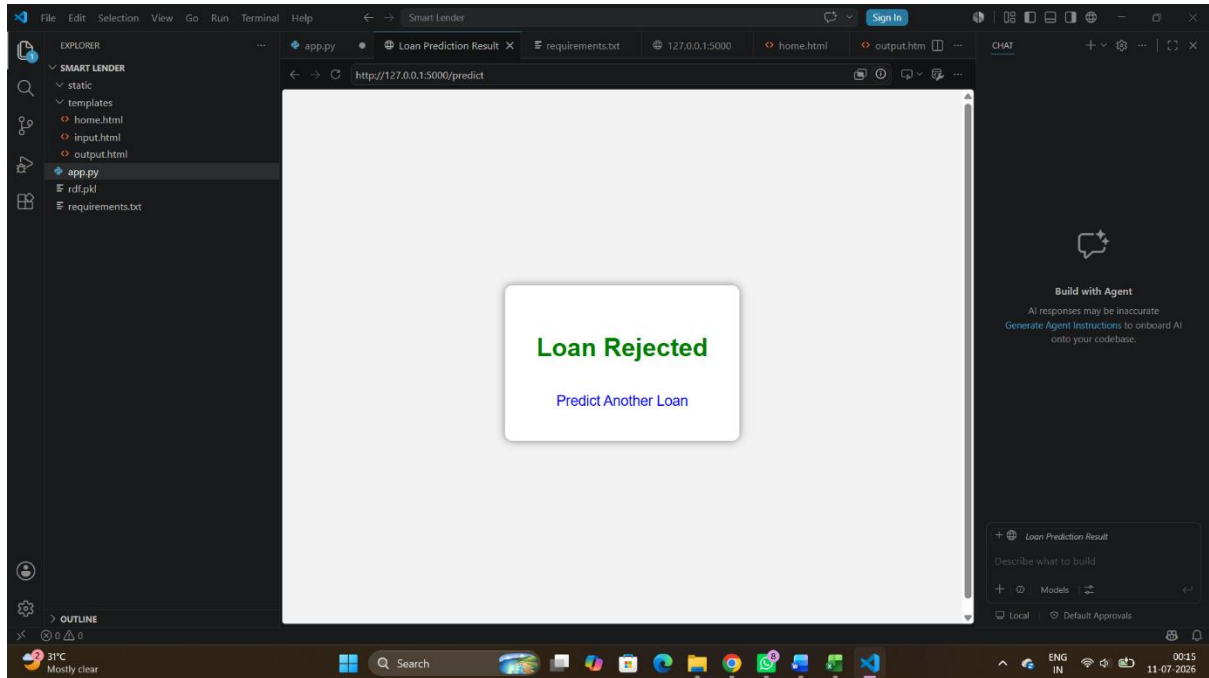
Activity 4.1: Designed the user interface using HTML and CSS to collect applicant information.



Activity 4.2: Connected the frontend with the Flask backend and displayed loan prediction results dynamically.

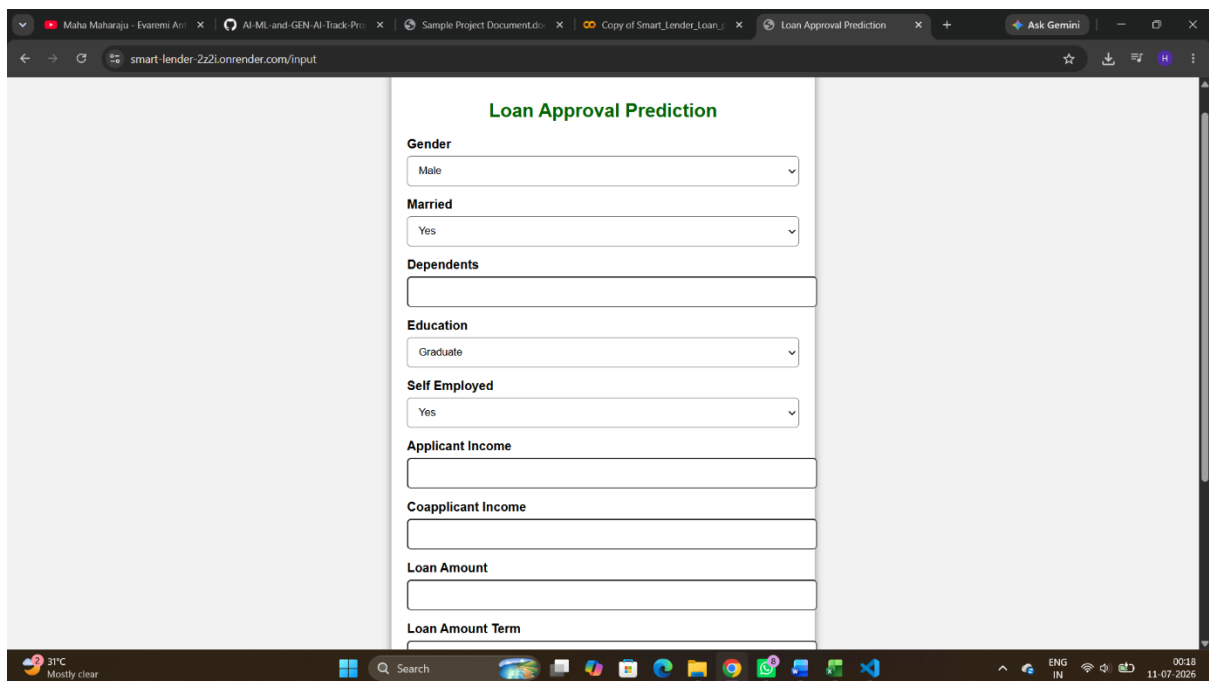


Activity 4.3: Tested the application with different sample inputs to verify accurate predictions.



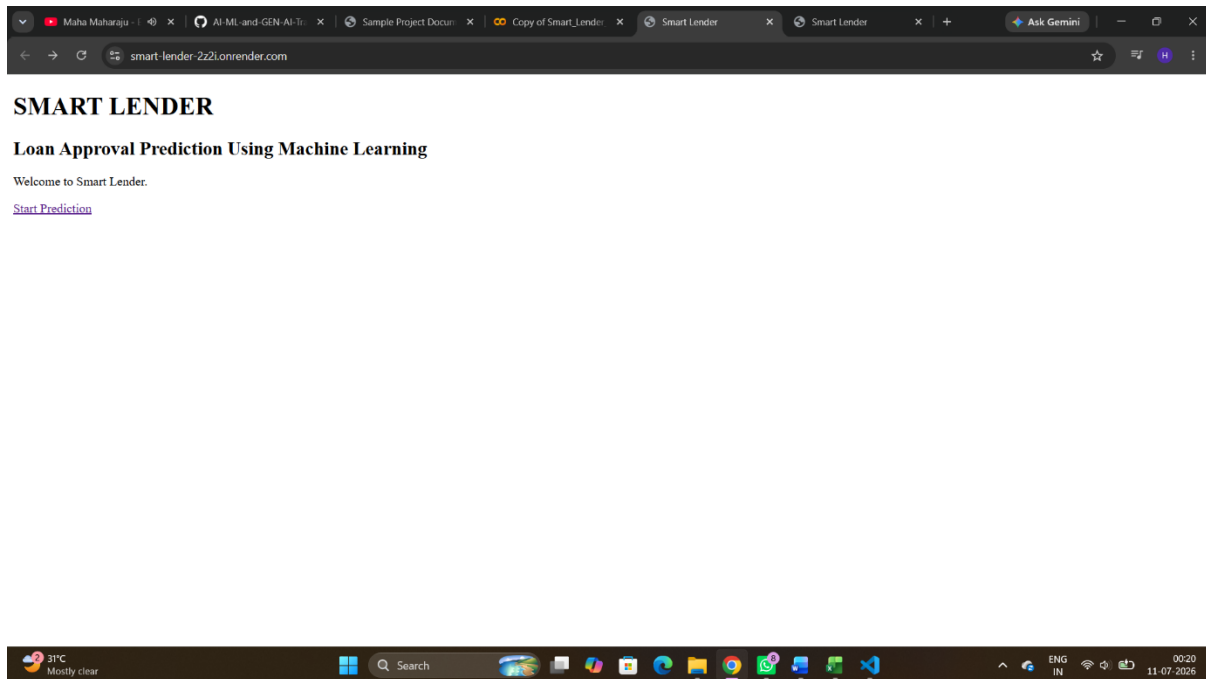
Milestone 5: Deployment and Testing

Activity 5.1: Tested the complete Smart Lender application locally and verified all functionalities.

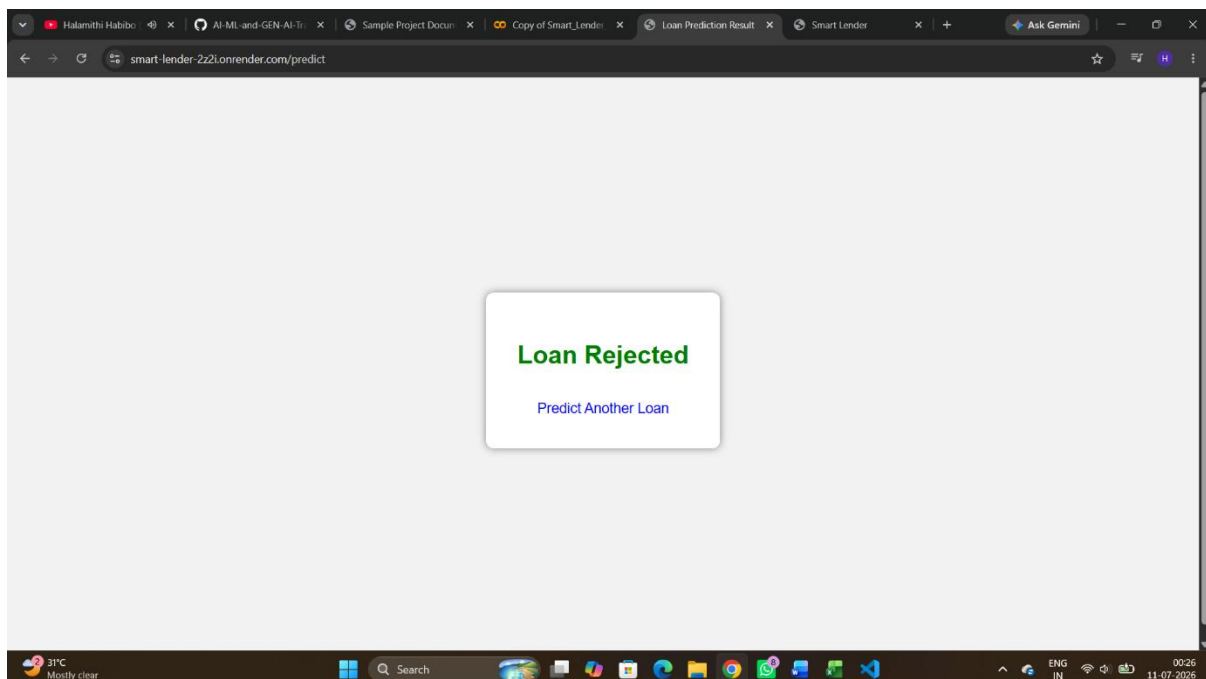


Activity 5.2: Successfully deployed the application on Render and generated a public deployment URL.

Link: <https://smart-lender-2z2i.onrender.com>

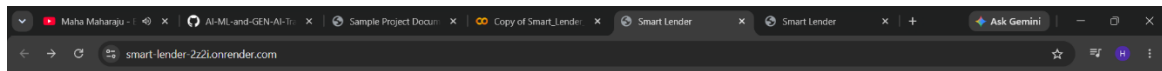


Activity 5.3: Validated the deployed application by testing multiple loan prediction scenarios and documented the results.



Exploring the Website's Web Pages:

Home / Generator Page:



SMART LENDER

Loan Approval Prediction Using Machine Learning

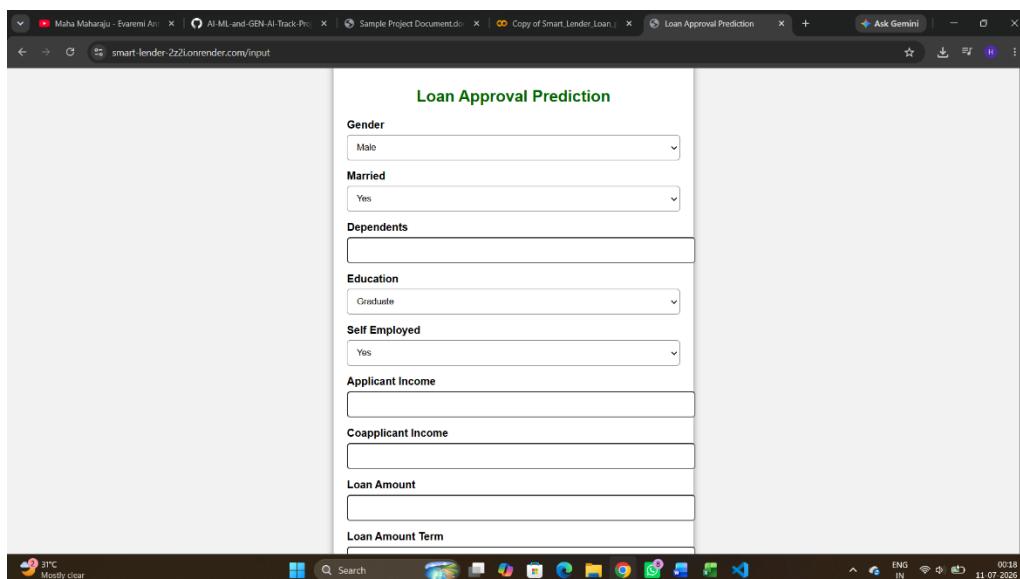
Welcome to Smart Lender.

[Start Prediction](#)



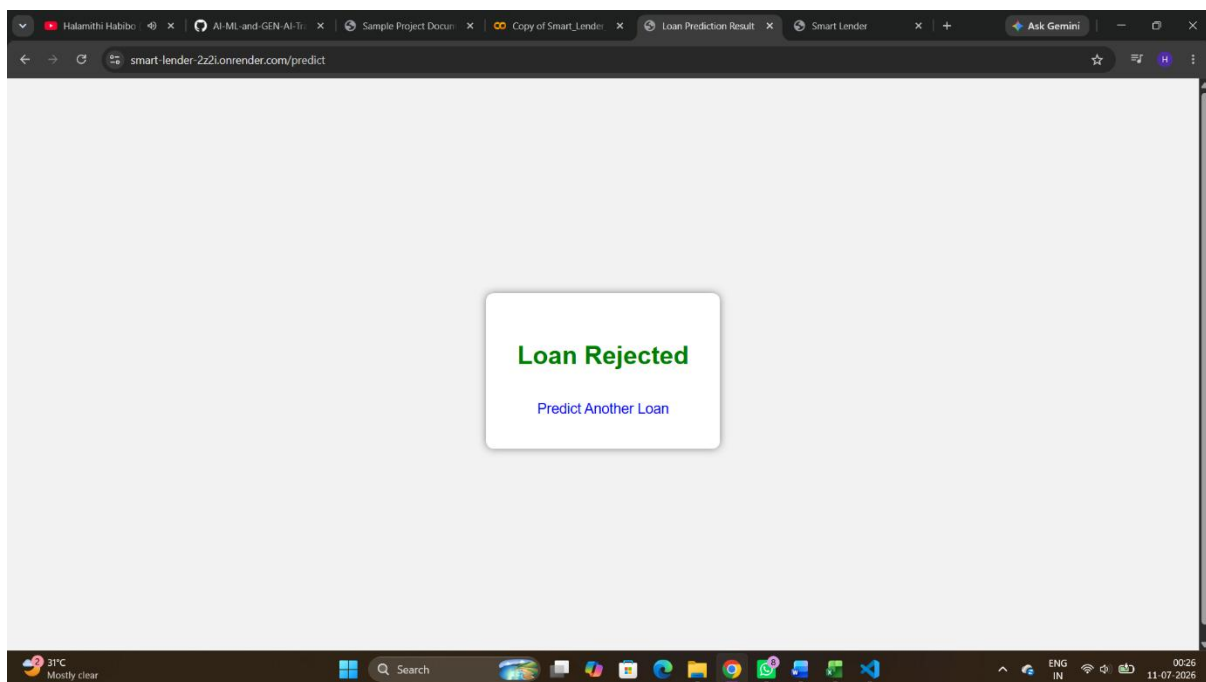
Description: The Home Page is the entry point of the Smart Lender application. It introduces the project as a Loan Approval Prediction System using Machine Learning and provides a simple, user-friendly interface. The page contains a "Start Prediction" button that redirects users to the loan application form, where they can enter applicant details and receive a loan approval prediction. This page serves as the main navigation page for accessing the prediction functionality.

Input Section:

A screenshot of the 'Loan Approval Prediction' input form. The form is titled 'Loan Approval Prediction' in green text. It contains several input fields with labels: 'Gender' (dropdown menu showing 'Male'), 'Married' (dropdown menu showing 'Yes'), 'Dependents' (text input field), 'Education' (dropdown menu showing 'Graduate'), 'Self Employed' (dropdown menu showing 'Yes'), 'Applicant Income' (text input field), 'Coapplicant Income' (text input field), 'Loan Amount' (text input field), and 'Loan Amount Term' (text input field). The form is displayed within a web browser window, with the address bar showing 'smart-lender-2z2i.onrender.com/input'. The Windows taskbar is visible at the bottom of the screen.

Description: The Input Page allows users to enter the applicant's details required for loan approval prediction. The form includes fields such as Gender, Marital Status, Dependents, Education, Self-Employment Status, Applicant Income, Co-applicant Income, Loan Amount, Loan Amount Term, Credit History, and Property Area. After filling in all the required information, the user clicks the Predict button to submit the data to the machine learning model, which analyzes the inputs and predicts whether the loan is likely to be approved or rejected.

Results Section:



Description: The Results Page displays the prediction generated by the machine learning model based on the applicant's input details. After the user submits the loan application form, the system analyzes the information and predicts whether the loan is approved or rejected. In this example, the model predicted "Loan Rejected." The page also provides a "Predict Another Loan" option, allowing users to return to the input form and test different applicant details.

Conclusion: The Smart Lender project was successfully developed and deployed as a Loan Approval Prediction System using Machine Learning. The system collects applicant details through a user-friendly web interface and predicts whether a loan is likely to be approved or rejected. During the project, data preprocessing, model training, and evaluation were performed using Decision Tree, Random Forest, and XGBoost algorithms. The best-performing model was integrated into a Flask-based web application, and the application was successfully deployed on Render with a public URL. This project demonstrates the complete machine learning workflow, including data preprocessing, model training, model evaluation, web application development, and deployment.